

TEORETICKÁ INFORMATIKA

Algoritmy BFS a DFS

David Weber
weber3@spsejecna.cz



Rok 2025/26

Prohledávání do šířky

Prohledávání do šířky

- Základním stavebním kamenem všech grafových algoritmů je určitý způsob procházení grafu od zadaného počátečního vrcholu.

- Základním stavebním kamenem všech grafových algoritmů je určitý způsob procházení grafu od zadaného počátečního vrcholu.
- My se zaměříme na dvojici nejjednodušších:

- Základním stavebním kamenem všech grafových algoritmů je určitý způsob procházení grafu od zadaného počátečního vrcholu.
- My se zaměříme na dvojici nejjednodušších:
 - **Prohledávání do šířky**, zkráceně **BFS** (Breadth-first search) a

Prohledávání do šířky

- Základním stavebním kamenem všech grafových algoritmů je určitý způsob procházení grafu od zadaného počátečního vrcholu.
- My se zaměříme na dvojici nejjednodušších:
 - Prohledávání do šířky, zkráceně **BFS** (Breadth-first search) a
 - Prohledávání do hloubky, zkráceně **DFS** (Depth-first search).

Postup algoritmu BFS

Postup algoritmu BFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.

Postup algoritmu BFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.

Postup algoritmu BFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Vrchol v_0 označíme jako **otevřený**.

Postup algoritmu BFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Vrchol v_0 označíme jako **otevřený**.
- 4 Prozkoumáme sousedy otevřených vrcholů (z počátku tedy sousedy v_0) a označíme je jako **otevřené**.

Postup algoritmu BFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Vrchol v_0 označíme jako **otevřený**.
- 4 Prozkoumáme sousedy otevřených vrcholů (z počátku tedy sousedy v_0) a označíme je jako **otevřené**.
- 5 Vrcholy, u nichž jsme prozkoumávali sousedy, označíme jako **uzavřené**.

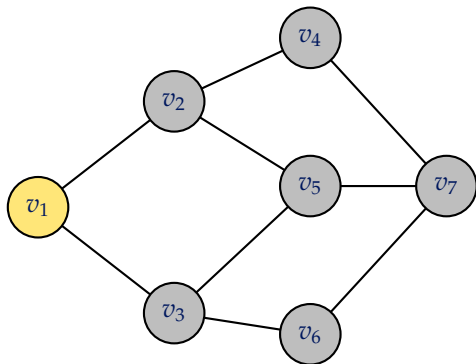
Postup algoritmu BFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Vrchol v_0 označíme jako **otevřený**.
- 4 Prozkoumáme sousedy otevřených vrcholů (z počátku tedy sousedy v_0) a označíme je jako **otevřené**.
- 5 Vrcholy, u nichž jsme prozkoumávali sousedy, označíme jako **uzavřené**.
- 6 Vrátime se ke 4. kroku a postup opakujeme.

Příklad

BFS – inicializace

Krok 0: start v v_1



● nenalezený

● otevřený

● uzavřený

— BFS strom

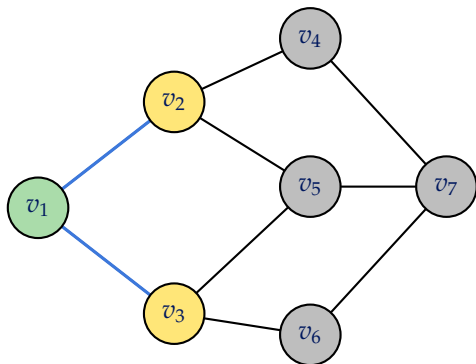
Otevřené vrcholy:

[v_1]

Příklad

BFS – po zpracování v_1

Krok 1: zpracujeme v_1



-  nenalezený
-  otevřený
-  uzavřený
-  BFS strom

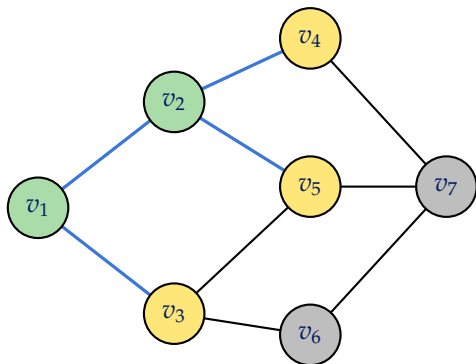
Otevřené vrcholy:

[v_2 , v_3]

Příklad

BFS – po zpracování v_2

Krok 2: zpracujeme v_2



-  nenalezený
-  otevřený
-  uzavřený
-  BFS strom

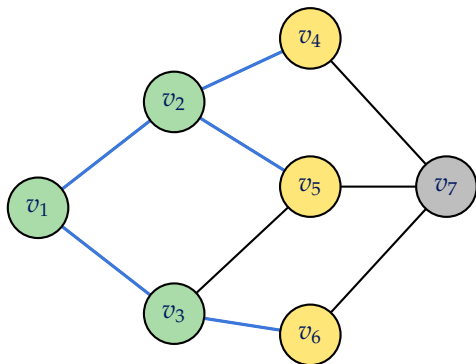
Otevřené vrcholy:

$[v_3, v_4, v_5]$

Příklad

BFS – po zpracování v_3

Krok 3: zpracujeme v_3



-  nenalezený
-  otevřený
-  uzavřený
-  BFS strom

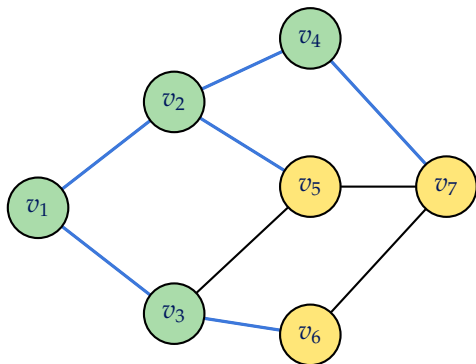
Otevřené vrcholy:

[v_4 , v_5 , v_6]

Příklad

BFS – po zpracování v_4

Krok 4: zpracujeme v_4



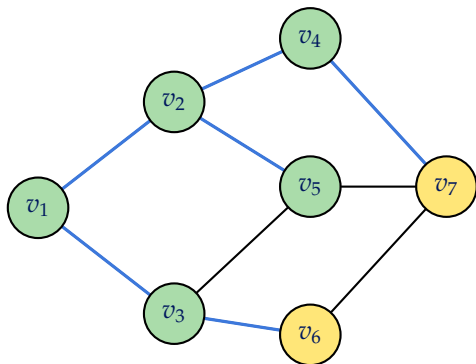
Otevřené vrcholy:

[v_5, v_6, v_7]

Příklad

BFS – po zpracování v_5

Krok 5: zpracujeme v_5



-  nenalezený
-  otevřený
-  uzavřený
-  BFS strom

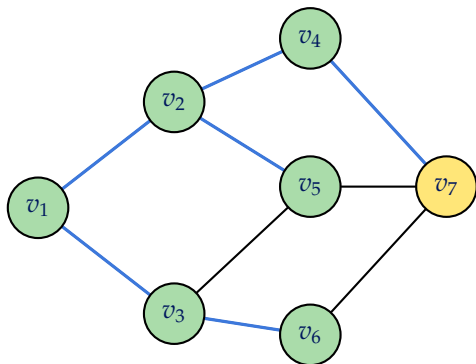
Otevřené vrcholy:

[v_6, v_7]

Příklad

BFS – po zpracování v_6

Krok 6: zpracujeme v_6



-  nenalezený
-  otevřený
-  uzavřený
-  BFS strom

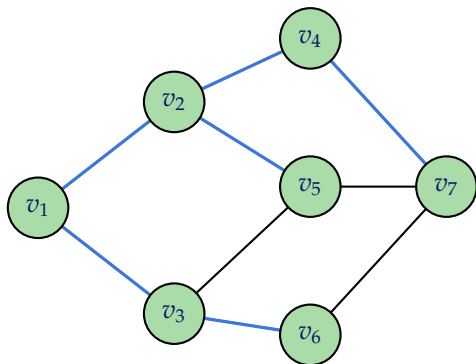
Otevřené vrcholy:

[v_7]

Příklad

BFS – konec

Krok 7: zpracujeme v_7



● nenalezený

● otevřený

● uzavřený

— BFS strom

Otevřené vrcholy:

[]

Algoritmus: BFS (prohledávání do šířky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$

```
1 foreach vrchol  $v \in V$  do
2    $\text{stav}(v) \leftarrow \textit{nenalezený}$ 
3    $D(v) \leftarrow \infty$ 
4  $\text{stav}(v_0) \leftarrow \textit{otevřený}, D(v_0) \leftarrow 0$ 
5 Založ frontu  $Q$  a přidej do ní vrchol  $v_0$ 
6 while fronta  $Q$  je neprázdná do
7   Odeber první vrchol ve frontě  $Q$  a označ jej  $v$ 
8   foreach sousední vrchol  $w$  vrcholu  $v$  do
9     if  $\text{stav}(w) = \textit{nenalezený}$  then
10       $\text{stav}(w) \leftarrow \textit{otevřený}$ 
11       $D(w) \leftarrow D(v) + 1$ 
12      Přidej  $w$  do fronty  $Q$ 
13    $\text{stav}(v) \leftarrow \textit{uzavřený}$ 
```

Implementace BFS v Pythonu

```
def bfs(graph, start):  
    state = {v: Stav.UNDISCOVERED for v in graph}  
    dist = {start: 0}  
    queue = deque([start])  
  
    state[start] = Stav.OPEN  
  
    while queue:  
        v = queue.popleft()  
        for u in graph[v]:  
            if state[u] == Stav.UNDISCOVERED:  
                state[u] = Stav.OPEN  
                dist[u] = dist[v] + 1  
                queue.append(u)  
        state[v] = Stav.CLOSED  
  
    return dist, state
```

Rozdělení vrcholů do vrstev

Rozdělení vrcholů do vrstev

- BFS prochází vrcholy „po vrstvách“

Rozdělení vrcholů do vrstev

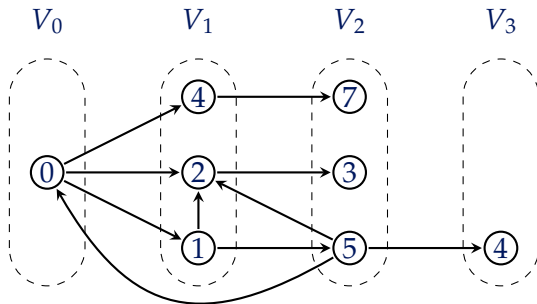
- BFS prochází vrcholy „po vrstvách“
 - Nejdříve zpracuje počáteční vrchol.
 - Pak zpracuje jeho sousedy.
 - Pak sousedy jeho sousedů atd.

Rozdělení vrcholů do vrstev

- BFS prochází vrcholy „po vrstvách“
 - Nejdříve zpracuje počáteční vrchol.
 - Pak zpracuje jeho sousedy.
 - Pak sousedy jeho sousedů atd.
- Tzn. kdykoliv uzavíráme vrchol ve vrstvě V_k , pak jeho neprozkoumané sousedy umístíme do vrstvy V_{k+1} .

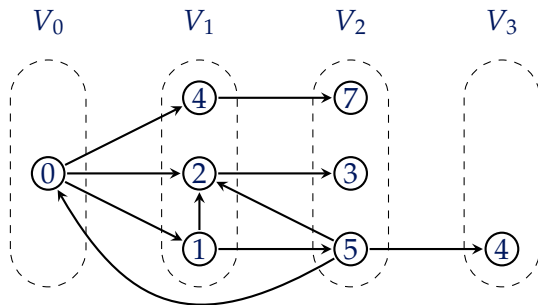
Rozdělení vrcholů do vrstev

- BFS prochází vrcholy „po vrstvách“
 - Nejdříve zpracuje počáteční vrchol.
 - Pak zpracuje jeho sousedy.
 - Pak sousedy jeho sousedů atd.
- Tzn. kdykoliv uzavíráme vrchol ve vrstvě V_k , pak jeho neprozkoumané sousedy umístíme do vrstvy V_{k+1} .



Rozdělení vrcholů do vrstev

- BFS prochází vrcholy „po vrstvách“
 - Nejdříve zpracuje počáteční vrchol.
 - Pak zpracuje jeho sousedy.
 - Pak sousedy jeho sousedů atd.
- Tzn. kdykoliv uzavíráme vrchol ve vrstvě V_k , pak jeho neprozkoumané sousedy umístíme do vrstvy V_{k+1} .



- Pro libovolný vrchol v ve vrstvě V_k tedy platí, že $D(v) = k$.

Pole vzdáleností D

Pole vzdáleností D

Tvrzení

Nechť je dán graf $G = (V, E)$, počáteční vrchol $v_0 \in V$ a libovolný vrchol $v \in V$. Pak na konci výpočtu BFS platí $D(v) = d(v, v_0)$, nebo-li $D(v)$ odpovídá vzdálenosti vrcholu v od počátečního vrcholu v_0 .

Pole vzdáleností D

Tvrzení

Nechť je dán graf $G = (V, E)$, počáteční vrchol $v_0 \in V$ a libovolný vrchol $v \in V$. Pak na konci výpočtu BFS platí $D(v) = d(v, v_0)$, nebo-li $D(v)$ odpovídá vzdálenosti vrcholu v od počátečního vrcholu v_0 .

Mějme libovolný vrchol v .

Pole vzdáleností D

Tvrzení

Nechť je dán graf $G = (V, E)$, počáteční vrchol $v_0 \in V$ a libovolný vrchol $v \in V$. Pak na konci výpočtu BFS platí $D(v) = d(v, v_0)$, nebo-li $D(v)$ odpovídá vzdálenosti vrcholu v od počátečního vrcholu v_0 .

Mějme libovolný vrchol v .

- Pokud je vrchol v nedosažitelný z vrcholu v_0 , pak $D(v) = d(v, v_0) = \infty$.

Pole vzdáleností D

Tvrzení

Nechť je dán graf $G = (V, E)$, počáteční vrchol $v_0 \in V$ a libovolný vrchol $v \in V$. Pak na konci výpočtu BFS platí $D(v) = d(v, v_0)$, nebo-li $D(v)$ odpovídá vzdálenosti vrcholu v od počátečního vrcholu v_0 .

Mějme libovolný vrchol v .

- Pokud je vrchol v nedosažitelný z vrcholu v_0 , pak $D(v) = d(v, v_0) = \infty$.
- Pokud je v dosažitelný, pak si rozdělíme zdůvodnění na dvě části:

$$\overbrace{D(v) \geq d(v, v_0)}^{1. \text{ část}} \quad \text{a} \quad \underbrace{D(v) \leq d(v, v_0)}_{2. \text{ část}}$$

Zdůvodnění $D(v) \geq d(v, v_0)$

Zdůvodnění $D(v) \geq d(v, v_0)$

- Pokud je v dosažitelný, algoritmus BFS jej musí během výpočtu nalézt.

Zdůvodnění $D(v) \geq d(v, v_0)$

- Pokud je v dosažitelný, algoritmus BFS jej musí během výpočtu nalézt.
- $\implies v$ musí být zařazený do nějaké vrstvy V_k .

Zdůvodnění $D(v) \geq d(v, v_0)$

- Pokud je v dosažitelný, algoritmus BFS jej musí během výpočtu nalézt.
- $\implies v$ musí být zařazený do nějaké vrstvy V_k .
- $\implies D(v) = k$.

Zdůvodnění $D(v) \geq d(v, v_0)$

- Pokud je v dosažitelný, algoritmus BFS jej musí během výpočtu nalézt.
- $\implies v$ musí být zařazený do nějaké vrstvy V_k .
- $\implies D(v) = k$.
- $\implies$ Do vrcholu v musí existovat nějaká cesta délky k , přičemž $d(v, v_0)$ je délka nejkratší z nich.

Zdůvodnění $D(v) \geq d(v, v_0)$

- Pokud je v dosažitelný, algoritmus BFS jej musí během výpočtu nalézt.
- $\implies v$ musí být zařazený do nějaké vrstvy V_k .
- $\implies D(v) = k$.
- $\implies$ Do vrcholu v musí existovat nějaká cesta délky k , přičemž $d(v, v_0)$ je délka nejkratší z nich.
- $\implies D(v) = k \geq d(v, v_0)$.

Zdůvodnění $D(v) \leq d(v, v_0)$

Zdůvodnění $D(v) \leq d(v, v_0)$

- Předpokládejme pro spor, že existují nějaké *špatné* vrcholy, pro které platí $D(u) > d(u, v_0)$.

Zdůvodnění $D(v) \leq d(v, v_0)$

- Předpokládejme pro spor, že existují nějaké *špatné* vrcholy, pro které platí $D(u) > d(u, v_0)$.
- Vybereme si **z nich** takový vrchol s , jehož skutečná vzdálenost $d(s, v_0)$ je nejmenší.
- Tento vrchol musí algoritmus BFS objevit (otevřít) přes nějakého předchůdce, označme jej p .

Zdůvodnění $D(v) \leq d(v, v_0)$

- Předpokládejme pro spor, že existují nějaké *špatné* vrcholy, pro které platí $D(u) > d(u, v_0)$.
- Vybereme si **z nich** takový vrchol s , jehož skutečná vzdálenost $d(s, v_0)$ je nejmenší.
- Tento vrchol musel algoritmus BFS objevit (otevřít) přes nějakého předchůdce, označme jej p .
- Protože

$$d(p, v_0) = d(s, v_0) - 1 < d(s, v_0),$$

pak p musí být *dobrý* vrchol (vzhledem k volbě vrcholu s)
 $\implies D(p) = d(p, v_0)$

Zdůvodnění $D(v) \leq d(v, v_0)$

- Předpokládejme pro spor, že existují nějaké *špatné* vrcholy, pro které platí $D(u) > d(u, v_0)$.
- Vybereme si **z nich** takový vrchol s , jehož skutečná vzdálenost $d(s, v_0)$ je nejmenší.
- Tento vrchol musel algoritmus BFS objevit (otevřít) přes nějakého předchůdce, označme jej p .
- Protože

$$d(p, v_0) = d(s, v_0) - 1 < d(s, v_0),$$

pak p musí být *dobrý* vrchol (vzhledem k volbě vrcholu s)

$$\implies D(p) = d(p, v_0)$$

- Vrchol s musel být objeven jako následník p a musel tedy padnout do vrstvy

$$D(s) = d(p, v_0) + 1$$

a tedy $D(s) = d(s, v_0)$.

Zdůvodnění $D(v) \leq d(v, v_0)$

- Předpokládejme pro spor, že existují nějaké *špatné* vrcholy, pro které platí $D(u) > d(u, v_0)$.
- Vybereme si **z nich** takový vrchol s , jehož skutečná vzdálenost $d(s, v_0)$ je nejmenší.
- Tento vrchol musel algoritmus BFS objevit (otevřít) přes nějakého předchůdce, označme jej p .
- Protože

$$d(p, v_0) = d(s, v_0) - 1 < d(s, v_0),$$

pak p musí být *dobrý* vrchol (vzhledem k volbě vrcholu s)

$$\implies D(p) = d(p, v_0)$$

- Vrchol s musel být objeven jako následník p a musel tedy padnout do vrstvy

$$D(s) = d(p, v_0) + 1$$

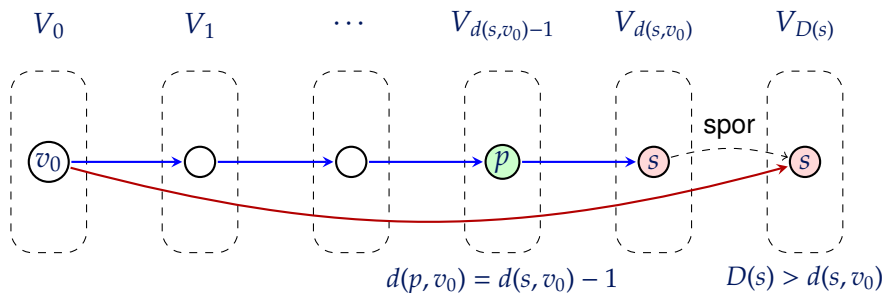
a tedy $D(s) = d(s, v_0)$.

- To je spor, neboť jsme předpokládali, že $D(s) > d(s, v_0)$.

Závěr tvrzení

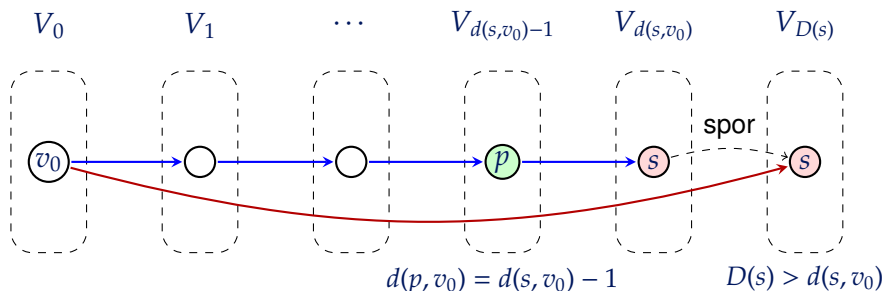
Závěr tvrzení

Znázornění situace v důkazu



Závěr tvrzení

Znázornění situace v důkazu



Celkově jsme zdůvodnili dvě nerovnosti:

$$D(v) \geq d(v, v_0) \quad \text{a} \quad D(v) \leq d(v, v_0),$$

z čehož již plyne $D(v) = d(v, v_0)$, což jsme chtěli.

Strom nejkratších cest

Strom nejkratších cest

- Průchodem algoritmu BFS vzniká tzv. **strom nejkratších cest**.

Strom nejkratších cest

- Průchodem algoritmu BFS vzniká tzv. **strom nejkratších cest**.

Již víme, že hodnota $D(v)$ odpovídá délce nejkratší cesty z v_0 do v .

Strom nejkratších cest

- Průchodem algoritmu BFS vzniká tzv. **strom nejkratších cest**.

Již víme, že hodnota $D(v)$ odpovídá délce nejkratší cesty z v_0 do v .

4

5

Strom nejkratších cest

- Průchodem algoritmu BFS vzniká tzv. **strom nejkratších cest**.

Již víme, že hodnota $D(v)$ odpovídá délce nejkratší cesty z v_0 do v .

- otevřením nového vrcholu vznikne nová hrana přidaná do tohoto stromu.

4

5

Strom nejkratších cest

- Průchodem algoritmu BFS vzniká tzv. **strom nejkratších cest**.

Již víme, že hodnota $D(v)$ odpovídá délce nejkratší cesty z v_0 do v .

- otevřením nového vrcholu vznikne nová hrana přidaná do tohoto stromu.
- Protože uzavřené vrcholy již znovu neotvíráme, nemůže nám vzniknout takovém grafu kružnice.

4

5

Časová a prostorová složitost BFS

Časová a prostorová složitost BFS

- Předpokládejme, že graf reprezentujeme pomocí seznamů sousedů.

Časová a prostorová složitost BFS

- Předpokládejme, že graf reprezentujeme pomocí seznamů sousedů.
- Dále budeme značit $n = |V|$ a $m = |E|$.

Časová a prostorová složitost BFS

- Předpokládejme, že graf reprezentujeme pomocí seznamů sousedů.
- Dále budeme značit $n = |V|$ a $m = |E|$.

Tvrzení

Algoritmus BFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

Časová a prostorová složitost BFS

- Předpokládejme, že graf reprezentujeme pomocí seznamů sousedů.
- Dále budeme značit $n = |V|$ a $m = |E|$.

Tvrzení

Algoritmus BFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Inicializace algoritmu trvá n -kroků.

Časová a prostorová složitost BFS

- Předpokládejme, že graf reprezentujeme pomocí seznamů sousedů.
- Dále budeme značit $n = |V|$ a $m = |E|$.

Tvrzení

Algoritmus BFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Inicializace algoritmu trvá n -kroků.
- Každý vrchol otevřeme a uzavřeme nejvýše jednou $\implies$ vnější cyklus proběhne maximálně n -krát.

Časová a prostorová složitost BFS

- Předpokládejme, že graf reprezentujeme pomocí seznamů sousedů.
- Dále budeme značit $n = |V|$ a $m = |E|$.

Tvrzení

Algoritmus BFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Inicializace algoritmu trvá n -kroků.
- Každý vrchol otevřeme a uzavřeme nejvýše jednou $\implies$ vnější cyklus proběhne maximálně n -krát.
 - V každé iteraci spotřebuje konstantní čas na svou reži a dále konstantní čas na každého následníka.

Časová a prostorová složitost BFS

- Předpokládejme, že graf reprezentujeme pomocí seznamů sousedů.
- Dále budeme značit $n = |V|$ a $m = |E|$.

Tvrzení

Algoritmus BFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Inicializace algoritmu trvá n -kroků.
- Každý vrchol otevřeme a uzavřeme nejvýše jednou $\implies$ vnější cyklus proběhne maximálně n -krát.
 - V každé iteraci spotřebuje konstantní čas na svou režii a dále konstantní čas na každého následníka.
- Tedy celkově

$$O\left(n + \sum_{i=1}^n \deg v_i\right) = O(n + m),$$

kde $v_i \in V$.

Časová a prostorová složitost BFS

- Protože využíváme seznam sousedů pro reprezentaci grafu, tak určitě spotřebujeme alespoň $O(n + m)$ paměti.

- Protože využíváme seznam sousedů pro reprezentaci grafu, tak určitě spotřebujeme alespoň $O(n + m)$ paměti.
- Zároveň spotřebujeme lineárně dlouhý prostor pro **uchování stavů vrcholů** a **seznam vzdáleností**.

Časová a prostorová složitost BFS

- Protože využíváme seznam sousedů pro reprezentaci grafu, tak určitě spotřebujeme alespoň $O(n + m)$ paměti.
- Zároveň spotřebujeme lineárně dlouhý prostor pro **uchování stavů vrcholů** a **seznam vzdáleností**.
- Celkově spotřebujeme

$$O(\overbrace{n}^{\text{stavy}} + \underbrace{n}_{\text{seznam } D} + \overbrace{n + m}^{\text{seznamy sousedů}}) = O(3n + m) = O(n + m)$$

Prohledávání do hloubky

- Postup algoritmu bude podobný jako u BFS, ale vrcholy budeme zpracovávat rekurzivně.

- Postup algoritmu bude podobný jako u BFS, ale vrcholy budeme zpracovávat rekurzivně.

- Postup algoritmu bude podobný jako u BFS, ale vrcholy budeme zpracovávat rekurzivně.
- Později uvidíme, že časová i prostorová složitost vyjde stejně.

Postup algoritmu DFS

Postup algoritmu DFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.

Postup algoritmu DFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.

Postup algoritmu DFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Na vrchol v_0 aplikujeme proceduru níže

Postup algoritmu DFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Na vrchol v_0 aplikujeme proceduru níže

Procedura (vstupní vrchol v)

Postup algoritmu DFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Na vrchol v_0 aplikujeme proceduru níže

Procedura (vstupní vrchol v)

- 1 Vrchol v označíme jako **otevřený**.

Postup algoritmu DFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Na vrchol v_0 aplikujeme proceduru níže

Procedura (vstupní vrchol v)

- 1 Vrchol v označíme jako **otevřený**.
- 2 Na každého **nenalezeného** souseda w vrcholu v aplikujeme **rekurzivně** stejnou proceduru.

Postup algoritmu DFS

- 1 Na vstupu máme graf $G = (V, E)$ a libovolný počáteční vrchol $v_0 \in V$.
- 2 Všechny vrcholy z V označíme jako **neprozkoumané**.
- 3 Na vrchol v_0 aplikujeme proceduru níže

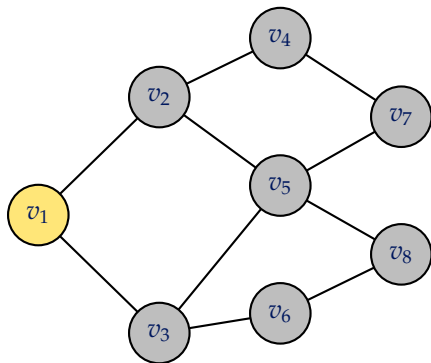
Procedura (vstupní vrchol v)

- 1 Vrchol v označíme jako **otevřený**.
- 2 Na každého **nenalezeného** souseda w vrcholu v aplikujeme **rekurzivně** stejnou proceduru.
- 3 Vrchol v označíme jako **uzavřený**.

Příklad

DFS – inicializace

Krok 0: start v v_1



nenalezený



otevřený



uzavřený



DFS strom

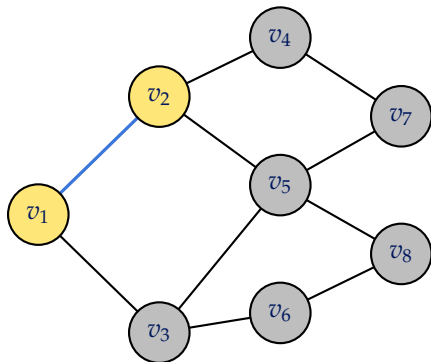
Otevřené vrcholy:

[v_1]

Příklad

DFS – po objevení v_2

Krok 1: z v_1 jdeme do v_2



nenalezený



otevřený



uzavřený



DFS strom

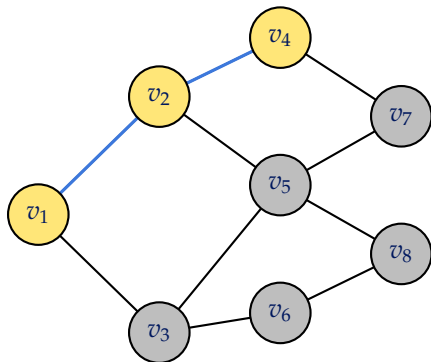
Otevřené vrcholy:

$[v_1, v_2]$

Příklad

DFS – po objevení v_4

Krok 2: z v_2 jdeme do v_4



nenalezený



otevřený



uzavřený



DFS strom

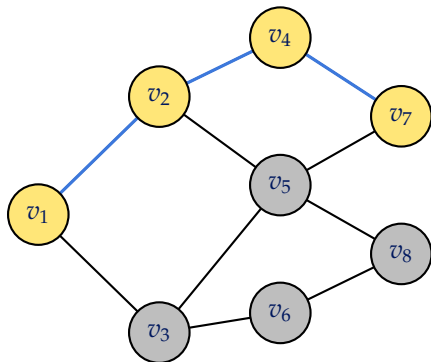
Otevřené vrcholy:

$[v_1, v_2, v_4]$

Příklad

DFS – po objevení v_7

Krok 3: z v_4 jdeme do v_7



nenalezený



otevřený



uzavřený



DFS strom

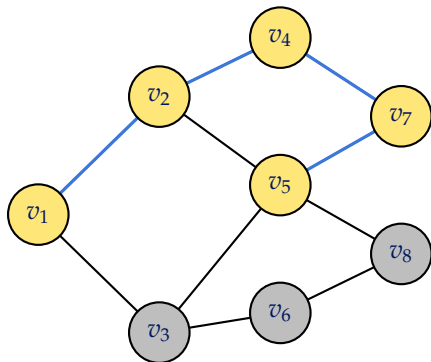
Otevřené vrcholy:

$[v_1, v_2, v_4, v_7]$

Příklad

DFS – po objevení v_5

Krok 4: z v_7 jdeme do v_5



nenalezený



otevřený



uzavřený



DFS strom

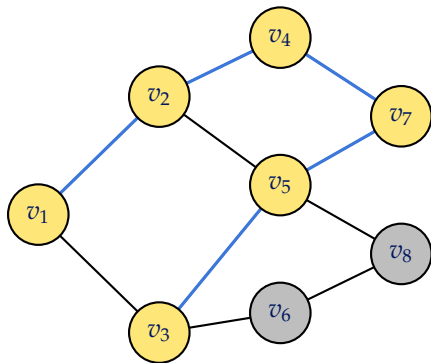
Otevřené vrcholy:

[v_1, v_2, v_4, v_7, v_5]

Příklad

DFS – po objevení v_3

Krok 5: z v_5 jdeme do v_3



nenalezený



otevřený



uzavřený



DFS strom

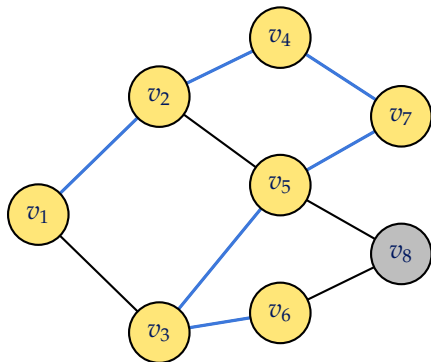
Otevřené vrcholy:

[$v_1, v_2, v_4, v_7, v_5, v_3$]

Příklad

DFS – po objevení v_6

Krok 6: z v_3 jdeme do v_6



nenalezený



otevřený



uzavřený



DFS strom

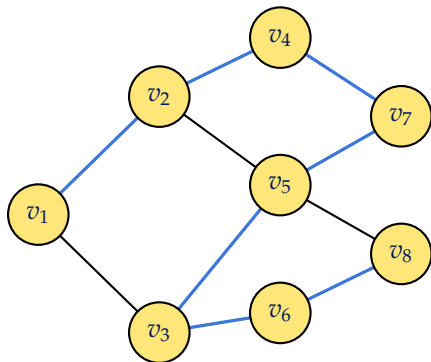
Otevřené vrcholy:

[$v_1, v_2, v_4, v_7, v_5, v_3, v_6$]

Příklad

DFS – po objevení v_8

Krok 7: z v_6 jdeme do v_8



nenalezený



otevřený



uzavřený



DFS strom

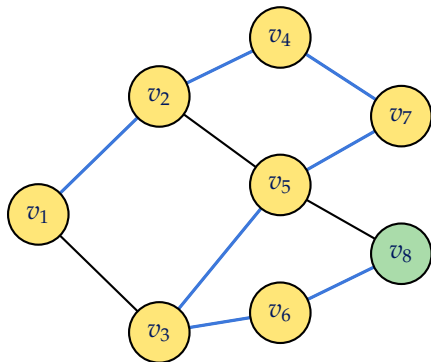
Otevřené vrcholy:

[v_1 , v_2 , v_4 , v_7 , v_5 , v_3 , v_6 , v_8]

Příklad

DFS – uzavření v_8

Krok 8: vracíme se z v_8



nenalezený



otevřený



uzavřený



DFS strom

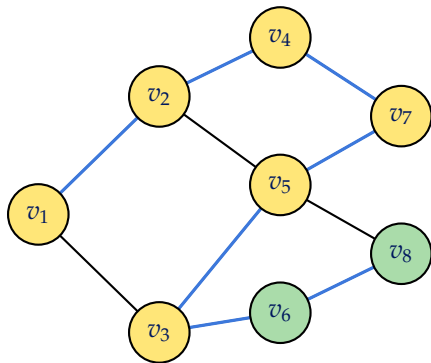
Otevřené vrcholy:

$[v_1, v_2, v_4, v_7, v_5, v_3, v_6]$

Příklad

DFS – uzavření v_6

Krok 9: vracíme se z v_6



nenalezený



otevřený



uzavřený



DFS strom

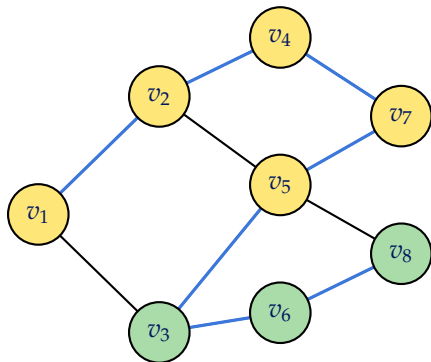
Otevřené vrcholy:

$[v_1, v_2, v_4, v_7, v_5, v_3]$

Příklad

DFS – uzavření v_3

Krok 10: vracíme se z v_3



nenalezený



otevřený



uzavřený



DFS strom

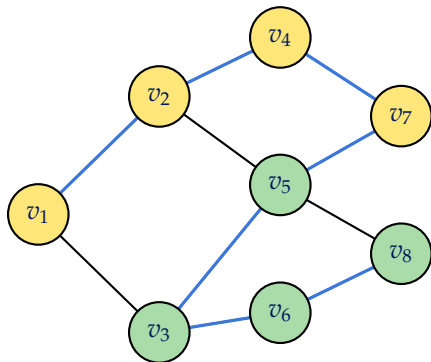
Otevřené vrcholy:

$[v_1, v_2, v_4, v_7, v_5]$

Příklad

DFS – uzavření v_5

Krok 11: vracíme se z v_5



nenalezený



otevřený



uzavřený



DFS strom

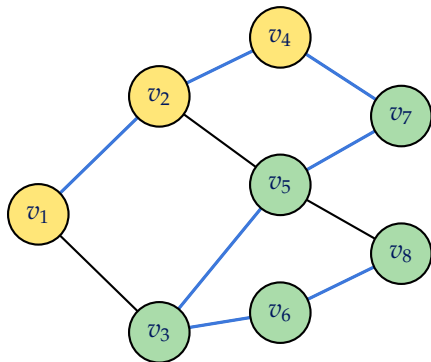
Otevřené vrcholy:

$[v_1, v_2, v_4, v_7]$

Příklad

DFS – uzavření v_7

Krok 12: vracíme se z v_7



nenalezený



otevřený



uzavřený



DFS strom

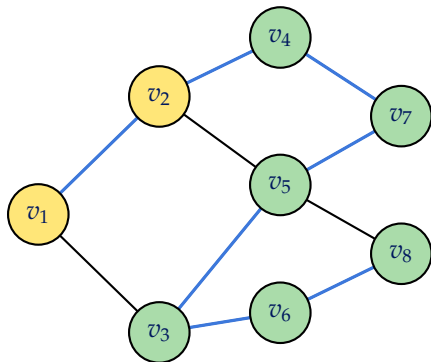
Otevřené vrcholy:

$[v_1, v_2, v_4]$

Příklad

DFS – uzavření v_4

Krok 13: vracíme se z v_4



nenalezený



otevřený



uzavřený



DFS strom

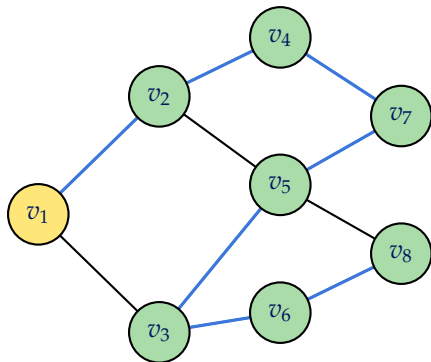
Otevřené vrcholy:

$[v_1, v_2]$

Příklad

DFS – uzavření v_2

Krok 14: vracíme se z v_2



nenalezený



otevřený



uzavřený



DFS strom

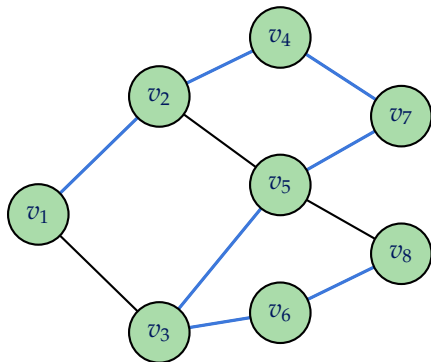
Otevřené vrcholy:

[v_1]

Příklad

DFS – konec

Krok 15: uzavřeme v_1



nenalezený



otevřený



uzavřený



DFS strom

Otevřené vrcholy:

[]

Pseudokód DFS

Algoritmus: DFS (prohledávání do hloubky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$

```
1 foreach vrchol  $v \in V$  do
2    $\text{stav}(v) \leftarrow \textit{nenalezený}$ 
3    $\text{in}(v), \text{out}(v) \leftarrow \textit{nedefinováno}$ 
4  $T \leftarrow 0$ 
5 Zavolej  $\text{DFS2}(v_0)$ 
6 Funkce  $\text{DFS2}(\text{vrchol } v \in V)$ 
7    $\text{stav}(v) \leftarrow \textit{otevřený}$ 
8    $T \leftarrow T + 1, \text{in}(v) \leftarrow T$ 
9   foreach sousední vrchol  $w$  vrcholu  $v$  do
10    if  $\text{stav}(w) = \textit{nenalezený}$  then
11      Zavolej  $\text{DFS2}(w)$ 
12    $\text{stav}(v) \leftarrow \textit{uzavřený}$ 
13    $T \leftarrow T + 1, \text{out}(v) \leftarrow T$ 
```

Implementace DFS v Pythonu

```
def dfs(graph):  
    state, tin, tout = {v: Stav.UNDISCOVERED for v in graph},  
        {}, {}  
    time = 0  
  
    def dfs2(v):  
        nonlocal time  
        state[v] = Stav.OPEN  
        time += 1  
        tin[v] = time  
  
        for u in graph[v]:  
            if state[u] == Stav.UNDISCOVERED: dfs2(u)  
  
        state[v] = Stav.CLOSED  
        time += 1  
        tout[v] = time  
  
    for v in graph:  
        if state[v] == Stav.UNDISCOVERED: dfs2(v)
```

Časová a prostorová složitost DFS

Časová a prostorová složitost DFS

Opět předpokládejme, že máme zadaný $G = (V, E)$ pomocí seznamu sousedů.

Časová a prostorová složitost DFS

Opět předpokládejme, že máme zadaný $G = (V, E)$ pomocí seznamu sousedů.

Tvrzení

Algoritmus DFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

Časová a prostorová složitost DFS

Opět předpokládejme, že máme zadaný $G = (V, E)$ pomocí seznamu sousedů.

Tvrzení

Algoritmus DFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Časová složitost plyne zcela triviálně ze skutečnosti, že DFS prochází vrcholy grafu pouze v jiném pořadí oproti BFS.

Časová a prostorová složitost DFS

Opět předpokládejme, že máme zadaný $G = (V, E)$ pomocí seznamu sousedů.

Tvrzení

Algoritmus DFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Časová složitost plyne zcela triviálně ze skutečnosti, že DFS prochází vrcholy grafu pouze v jiném pořadí oproti BFS.
- Dále spotřebujeme $O(n + m)$ paměti na reprezentaci grafu.

Časová a prostorová složitost DFS

Opět předpokládejme, že máme zadaný $G = (V, E)$ pomocí seznamu sousedů.

Tvrzení

Algoritmus DFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Časová složitost plyne zcela triviálně ze skutečnosti, že DFS prochází vrcholy grafu pouze v jiném pořadí oproti BFS.
- Dále spotřebujeme $O(n + m)$ paměti na reprezentaci grafu.
- Dále budeme potřebovat ještě lineární množství paměti na zásobník rekurzivního volání funkce DFS2 a na pomocné seznamy (stavy vrcholů, seznamy **in** a **out**).

Časová a prostorová složitost DFS

Opět předpokládejme, že máme zadaný $G = (V, E)$ pomocí seznamu sousedů.

Tvrzení

Algoritmus DFS doběhne v čase $O(n + m)$ a spotřebuje paměť $O(n + m)$.

- Časová složitost plyne zcela triviálně ze skutečnosti, že DFS prochází vrcholy grafu pouze v jiném pořadí oproti BFS.
- Dále spotřebujeme $O(n + m)$ paměti na reprezentaci grafu.
- Dále budeme potřebovat ještě lineární množství paměti na zásobník rekurzivního volání funkce DFS2 a na pomocné seznamy (stavy vrcholů, seznamy *in* a *out*).

$$O(\underbrace{2n}_{\text{seznamy in a out}} + \underbrace{n}_{\text{zásobník}} + \underbrace{n}_{\text{stavy}} + \underbrace{n + m}_{\text{seznamy sousedů}}) = O(n + m).$$

DFS strom

Stejně jako v případě BFS, i při průchodu grafem pomocí DFS vzniká **strom** jako podgraf původního grafu G .

DFS strom

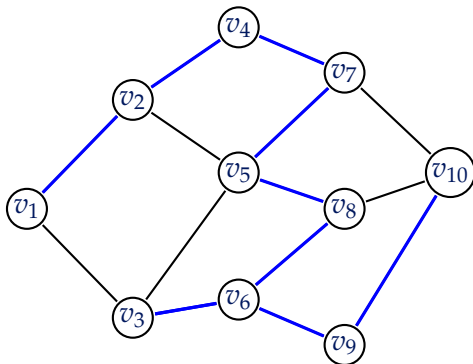
Stejně jako v případě BFS, i při průchodu grafem pomocí DFS vzniká **strom** jako podgraf původního grafu G .

Pozor! Nalezené cesty už nemusí
být nejkratší!

DFS strom

Stejně jako v případě BFS, i při průchodu grafem pomocí DFS vzniká **strom** jako podgraf původního grafu G .

Pozor! Nalezené cesty už nemusí být nejkratší!



Tvrzení

Nechť $G = (V, E)$ je graf a $u, v \in V$. Po průchodu grafu G pomocí DFS nastane vždy jedna z možností:

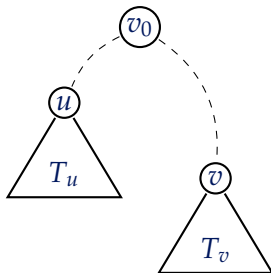
- 1 $\langle \text{in}(u), \text{out}(u) \rangle \cap \langle \text{in}(v), \text{out}(v) \rangle = \emptyset$,
- 2 $\langle \text{in}(u), \text{out}(u) \rangle \subseteq \langle \text{in}(v), \text{out}(v) \rangle$,
- 3 $\langle \text{in}(v), \text{out}(v) \rangle \subseteq \langle \text{in}(u), \text{out}(u) \rangle$.

Tvrzení v podstatě popisuje možnou vzájemnou pozici vrcholů v DFS stromě.

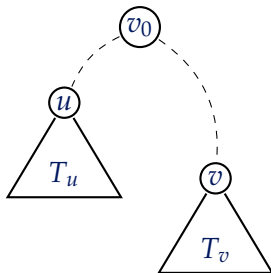
- **Bod 1.** Vrcholy u a v náležejí různým podstromům.
- **Body 2 a 3.** Jeden z vrcholů u, v náleží podstromu „druhého vrcholu“.

Případ $\langle \text{in}(u), \text{out}(u) \rangle \cap \langle \text{in}(v), \text{out}(v) \rangle = \emptyset$

Případ $\langle \text{in}(u), \text{out}(u) \rangle \cap \langle \text{in}(v), \text{out}(v) \rangle = \emptyset$



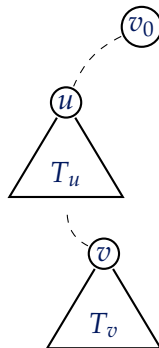
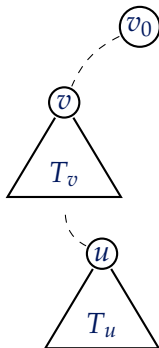
Případ $\langle \text{in}(u), \text{out}(u) \rangle \cap \langle \text{in}(v), \text{out}(v) \rangle = \emptyset$



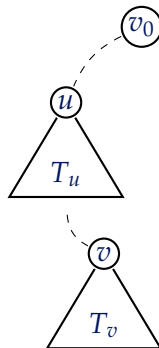
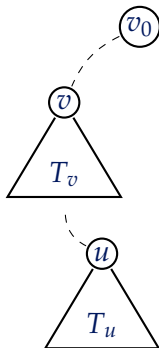
T_u, T_v jsou (pod)stromy v grafu G , kde po řadě vrcholy u a v jsou jejich kořeny.

Případ $\langle \text{in}(u), \text{out}(u) \rangle \subseteq \langle \text{in}(v), \text{out}(v) \rangle$

Případ $\langle \text{in}(u), \text{out}(u) \rangle \subseteq \langle \text{in}(v), \text{out}(v) \rangle$



Případ $\langle \text{in}(u), \text{out}(u) \rangle \subseteq \langle \text{in}(v), \text{out}(v) \rangle$

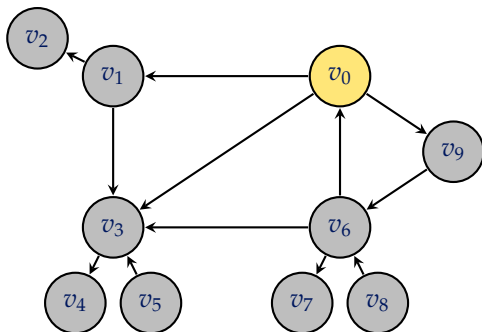


Obě situace jsou symetrické (vrcholy si jen prohodí roli).

Příklad

DFS – krok 1

Krok 1: objevíme v_0

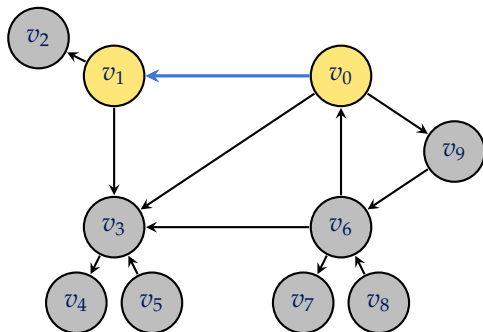


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	.	.
v_2	.	.
v_3	.	.
v_4	.	.
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 2

Krok 2: z v_0 jdeme do v_1

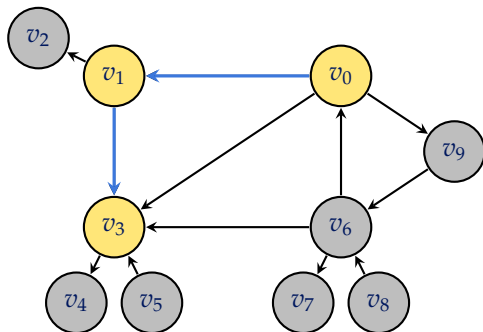


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	.
v_2	.	.
v_3	.	.
v_4	.	.
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 3

Krok 3: z v_1 jdeme do v_3

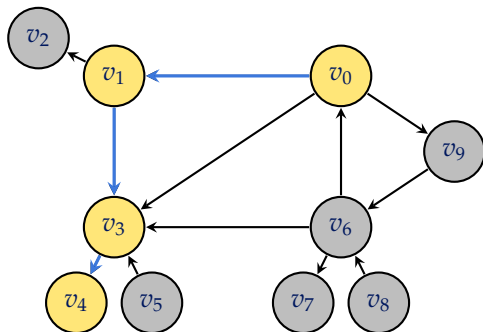


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	.
v_2	.	.
v_3	3	.
v_4	.	.
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 4

Krok 4: z v_3 jdeme do v_4

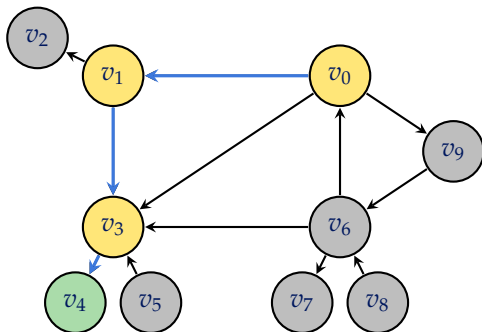


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	.
v_2	.	.
v_3	3	.
v_4	4	.
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 5

Krok 5: uzavřeme v_4

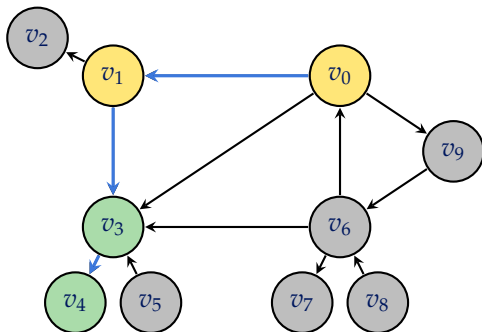


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	.
v_2	.	.
v_3	3	.
v_4	4	5
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 6

Krok 6: uzavřeme v_3

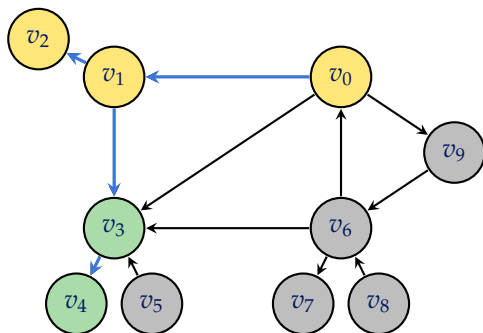


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	.
v_2	.	.
v_3	3	6
v_4	4	5
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 7

Krok 7: z v_1 jdeme do v_2

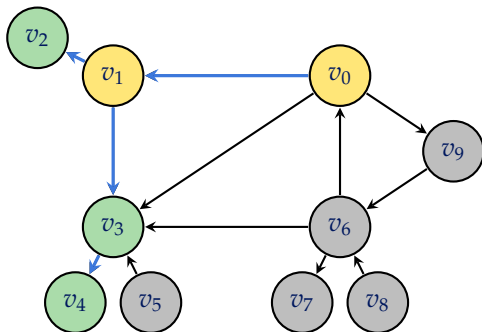


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	.
v_2	7	.
v_3	3	6
v_4	4	5
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 8

Krok 8: uzavřeme v_2

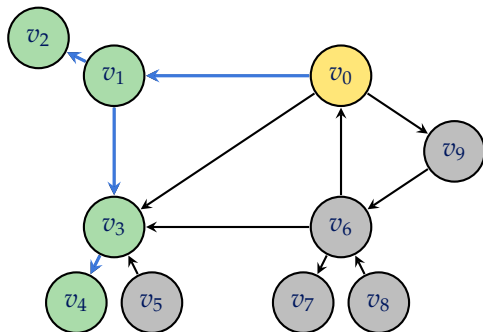


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	.
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 9

Krok 9: uzavřeme v_1

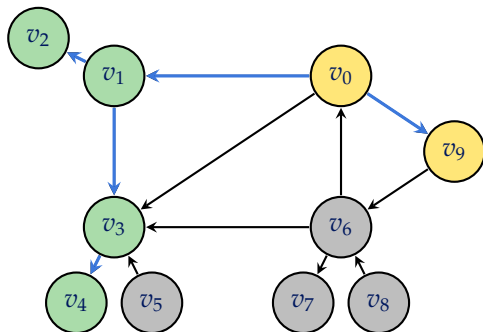


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	.	.

Příklad

DFS – krok 10

Krok 10: z v_0 jdeme do v_9

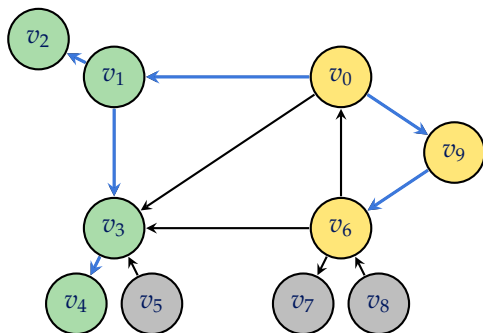


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	.	.
v_7	.	.
v_8	.	.
v_9	10	.

Příklad

DFS – krok 11

Krok 11: z v_9 jdeme do v_6

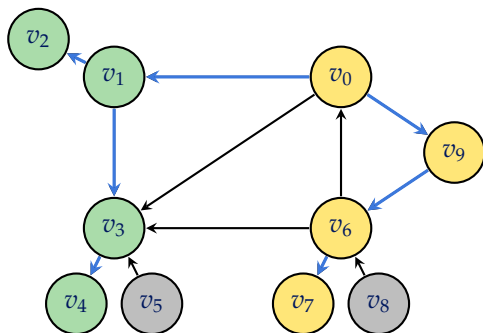


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	11	.
v_7	.	.
v_8	.	.
v_9	10	.

Příklad

DFS – krok 12

Krok 12: z v_6 jdeme do v_7

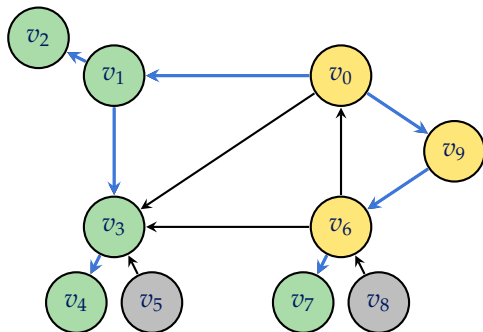


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	11	.
v_7	12	.
v_8	.	.
v_9	10	.

Příklad

DFS – krok 13

Krok 13: uzavřeme v_7

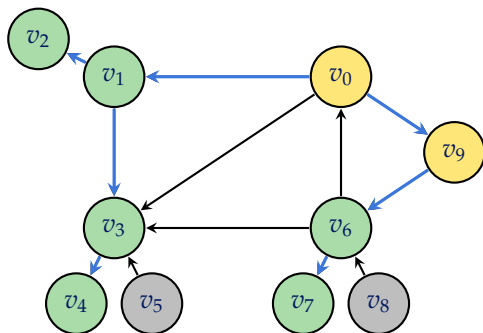


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	11	.
v_7	12	13
v_8	.	.
v_9	10	.

Příklad

DFS – krok 14

Krok 14: uzavřeme v_6

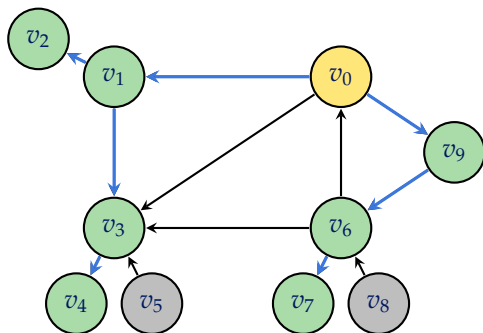


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	.
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	11	14
v_7	12	13
v_8	.	.
v_9	10	.

Příklad

DFS – krok 15

Krok 15: uzavřeme v_9

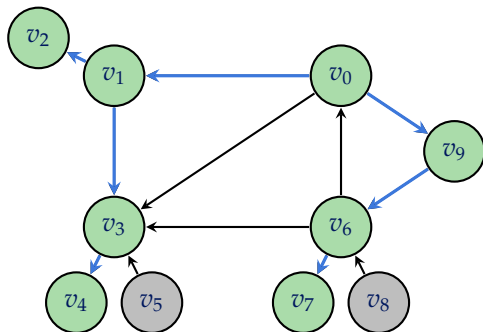


v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	·
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	·	·
v_6	11	14
v_7	12	13
v_8	·	·
v_9	10	15

Příklad

DFS – krok 16

Krok 16: uzavřeme v_0



v	$\text{in}(v)$	$\text{out}(v)$
v_0	1	16
v_1	2	9
v_2	7	8
v_3	3	6
v_4	4	5
v_5	.	.
v_6	11	14
v_7	12	13
v_8	.	.
v_9	10	15

Mosty v grafu

Mosty v grafu

Mosty v grafu

Typický případem užití DFS je např. hledání mostů v grafu.

BFS zde použít nelze!

Mosty v grafu

Typický případem užití DFS je např. hledání mostů v grafu.

BFS zde použít nelze!

Definice (Most)

Nechť $G = (V, E)$ je graf. Hrana $e \in E$ je **most**, pokud po jejím odebrání z grafu G vznikne graf $G' = (V', E')$ takový, že v něm existují vrcholy $u, v \in V'$, mezi nimiž neexistuje cesta.

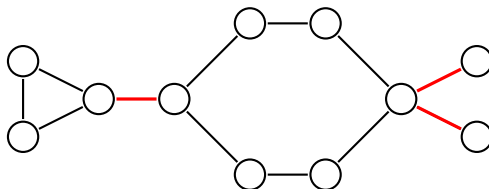
Mosty v grafu

Typický případem užití DFS je např. hledání mostů v grafu.

BFS zde použít nelze!

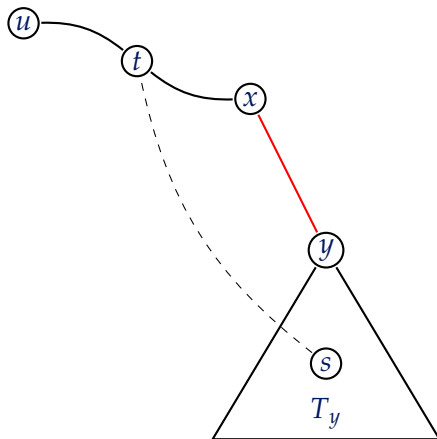
Definice (Most)

Nechť $G = (V, E)$ je graf. Hrana $e \in E$ je **most**, pokud po jejím odebrání z grafu G vznikne graf $G' = (V', E')$ takový, že v něm existují vrcholy $u, v \in V'$, mezi nimiž neexistuje cesta.

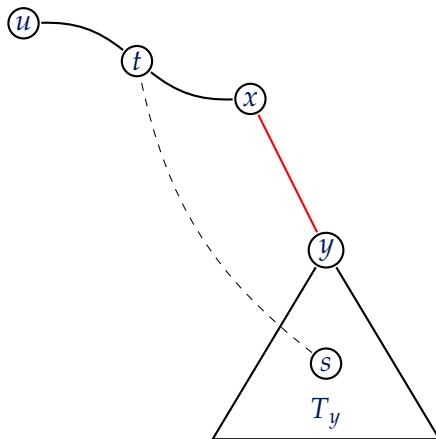


Idea algoritmu pro hledání mostů

Idea algoritmu pro hledání mostů



Idea algoritmu pro hledání mostů



Pokud během výpočtu nalezneme hranu z podstromu T_y , která vede do již otevřeného vrcholu t nacházejícího se v DFS stromě „nad“ vrchlem y , pak xy **nemůže být most!**

Idea algoritmu pro hledání mostů

Idea algoritmu pro hledání mostů

- Jak ale poznat, jestli t leží „nad“ vrcholem y ?

Idea algoritmu pro hledání mostů

- Jak ale poznat, jestli t leží „nad“ vrcholem y ?

⇒ To lze poznat podle hodnoty in ! V takovém případě totiž musí platit

$$\text{in}(t) < \text{in}(y).$$

Idea algoritmu pro hledání mostů

- Jak ale poznat, jestli t leží „nad“ vrcholem y ?

⇒ To lze poznat podle hodnoty in ! V takovém případě totiž musí platit

$$\text{in}(t) < \text{in}(y).$$

- Pro každý vrchol v si budeme uchovávat, jak „vysoko“ se z něj můžeme dostat v DFS stromu.

Idea algoritmu pro hledání mostů

- Jak ale poznat, jestli t leží „nad“ vrcholem y ?

⇒ To lze poznat podle hodnoty in ! V takovém případě totiž musí platit

$$\text{in}(t) < \text{in}(y).$$

- Pro každý vrchol v si budeme uchovávat, jak „vysoko“ se z něj můžeme dostat v DFS stromu.

To lze zjistit tak, že vezmeme minimum z in ů všech vrcholů, do který se můžeme dostat z podstromu y .

Další užití BFS a DFS

Další užití BFS a DFS

Možná využití BFS

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,
- procházení stavového prostoru po vrstvách,

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,
- procházení stavového prostoru po vrstvách,
- hledání řešení s minimálním počtem kroků.

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,
- procházení stavového prostoru po vrstvách,
- hledání řešení s minimálním počtem kroků.

Možná využití DFS

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,
- procházení stavového prostoru po vrstvách,
- hledání řešení s minimálním počtem kroků.

Možná využití DFS

- hledání souvislých komponent,

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,
- procházení stavového prostoru po vrstvách,
- hledání řešení s minimálním počtem kroků.

Možná využití DFS

- hledání souvislých komponent,
- topologické uspořádání,

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,
- procházení stavového prostoru po vrstvách,
- hledání řešení s minimálním počtem kroků.

Možná využití DFS

- hledání souvislých komponent,
- topologické uspořádání,
- hledání mostů a artikulací,

Možná využití BFS

- určení vzdáleností od počátečního vrcholu,
- hledání řešení s minimálním počtem kroků,
- procházení stavového prostoru po vrstvách,
- hledání řešení s minimálním počtem kroků.

Možná využití DFS

- hledání souvislých komponent,
- topologické uspořádání,
- hledání mostů a artikulací,
- analýza struktury grafu pomocí DFS stromu.

- MAREŠ, Martin a VALLA, Tomáš. *Průvodce labyrintem algoritmů*. Druhé vydání. CZ.NIC. Praha: CZ.NIC, z.s.p.o., 2022. ISBN 978-80-88168-63-8.
- CORMEN, Thomas H.; LEISERSON, Charles Eric; RIVEST, Ronald L. a STEIN, Clifford. *Introduction to algorithms*. Fourth edition. Cambridge: The MIT Press, 2022. ISBN 978-0-262-04630-5.